

Лабораторная работа №2. Условия, диапазоны и коллекции в Kotlin

Цель:

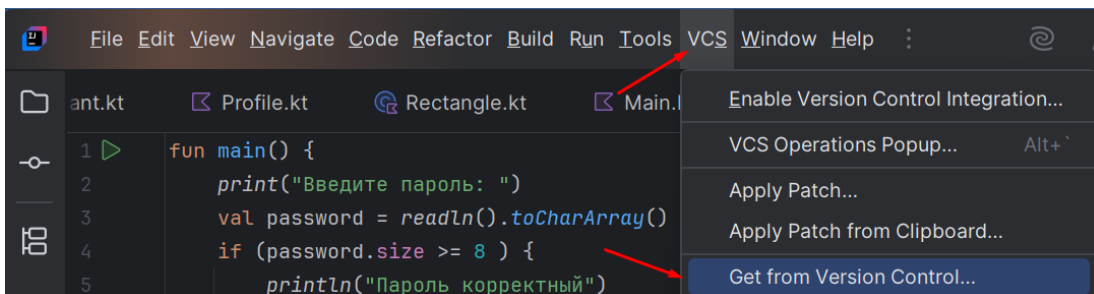
Изучить основы логики программирования на Kotlin — условные конструкции, диапазоны, а также сделать первые шаги в работе с коллекциями и системой контроля версий Git.

Задачи:

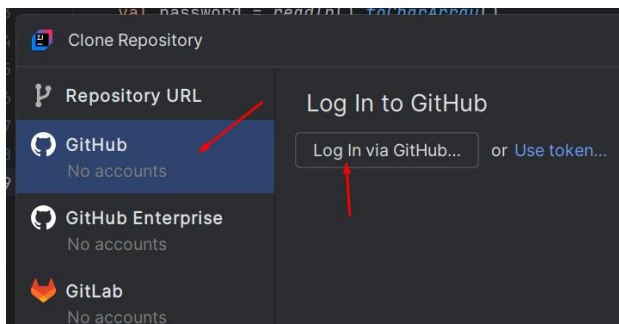
- Научиться инициализировать новый проект с Git-репозиторием.
- Создать точку входа (fun main) и выполнить базовые коммиты.
- Изучить конструкции if, when и диапазоны .., in.
- Освоить цикл repeat и базовую работу с коллекциями.
- Познакомиться с символьным типом данных (Char).

Шаг 1. Создание нового проекта в IntelliJ IDEA и репозитория в Git

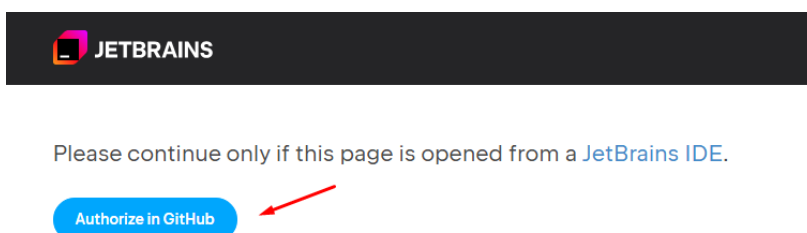
Запустите **IntelliJ IDEA**. По умолчанию у вас откроется ваш предыдущий проект. Давайте создадим новый и создадим репозиторий на **GitHub**. Войдите в свою учетную запись на **GitHub**. Затем перейдите в меню **VCS – Get from Version Control**:



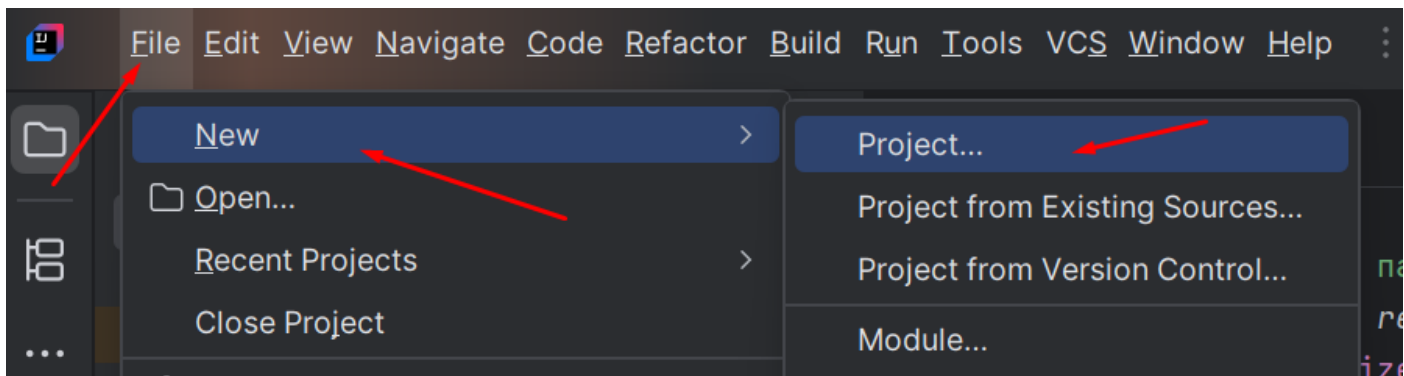
GitHub – Log In via GitHub:



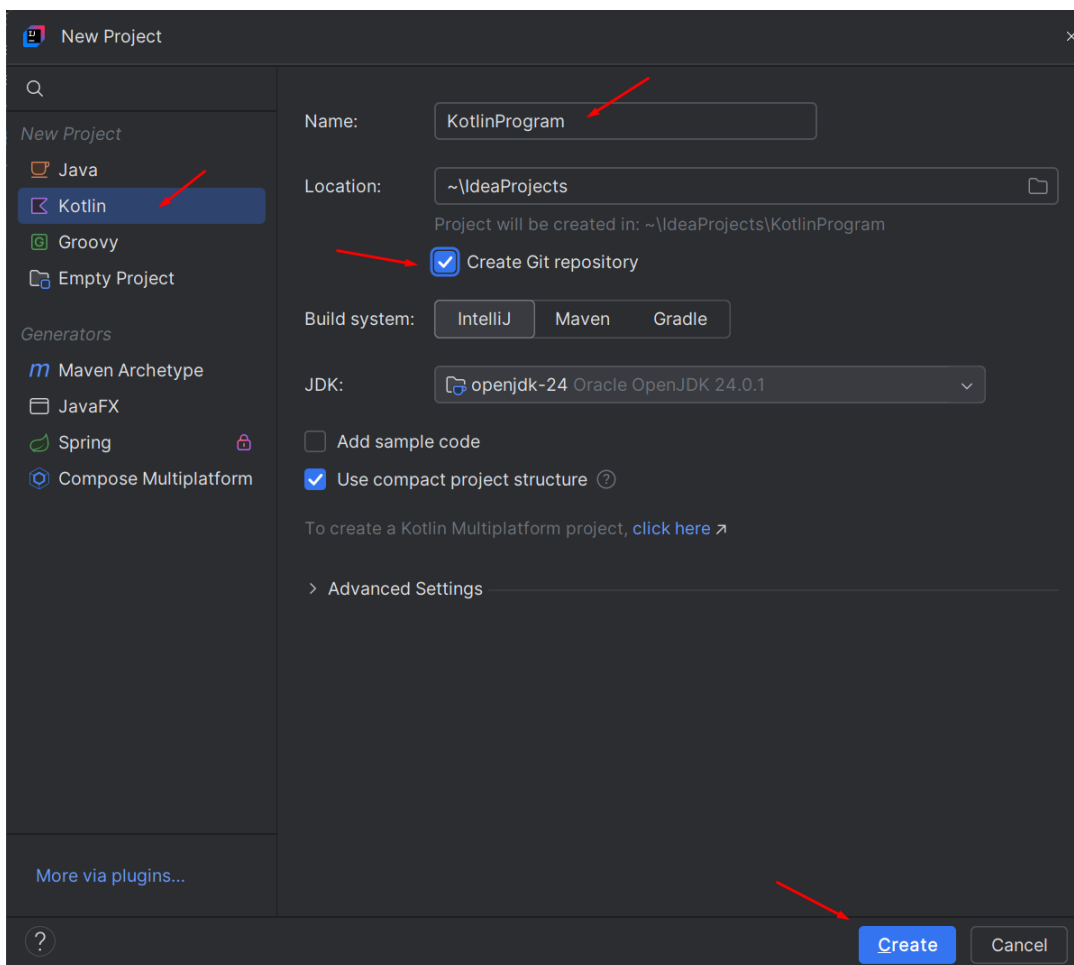
Нажимаете авторизоваться:



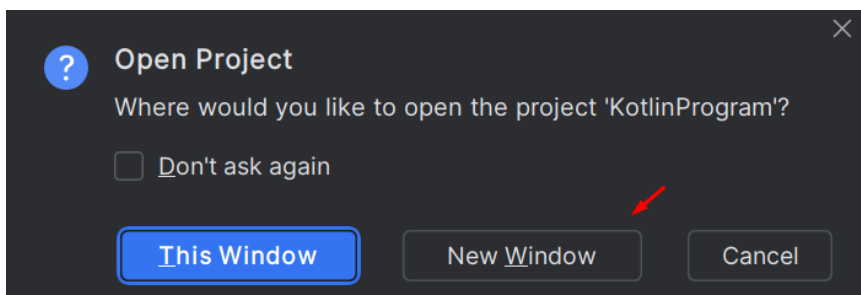
Далее нажмите **File – New – Project**:



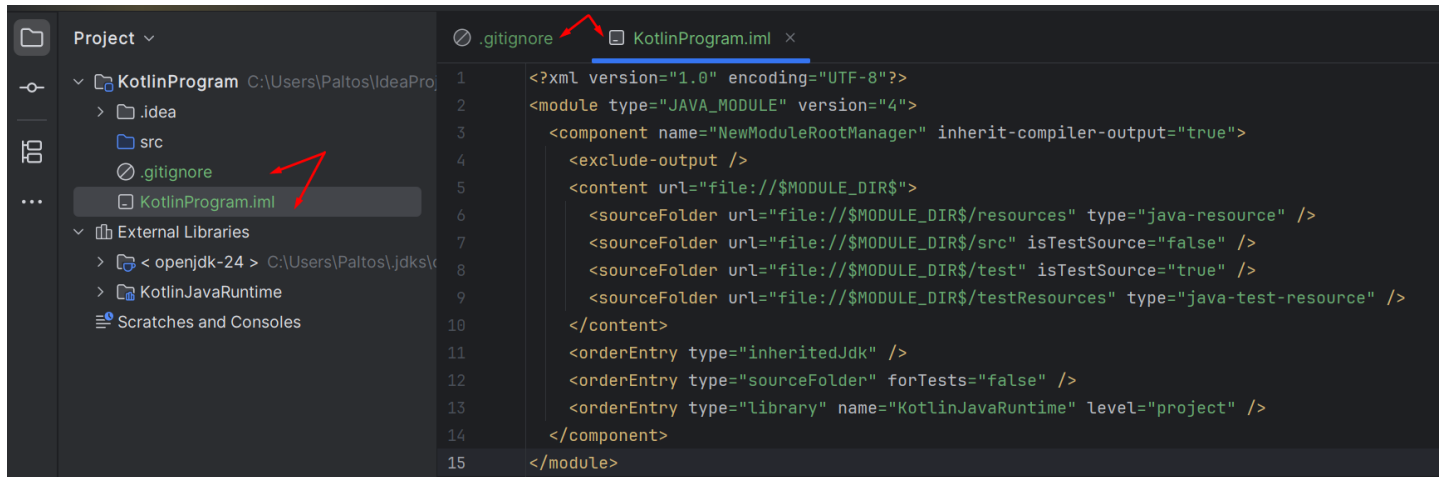
В окне создания проекта убедитесь, что выбрать язык **Kotlin**, название проекта придумайте **свое**, поставьте галочку **Create Git repository** и нажмите **Create**:



Выберите в **новом** или **текущем** окне:



Теперь, когда у вас создастся проект, обратите внимание что появились два новых файла **.gitignore** и **KotlinProgram.iml**:



1. .gitignore

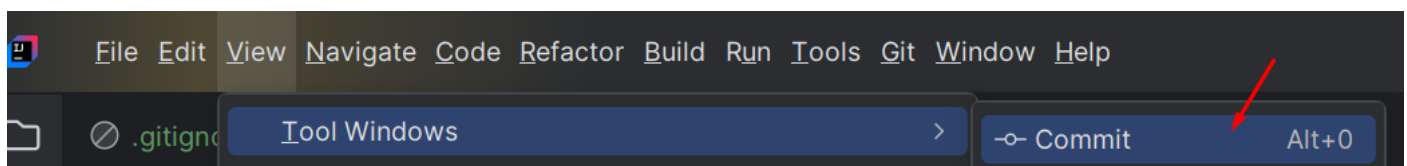
- Файл для Git, указывающий, какие файлы и папки не нужно отслеживать (не добавлять в репозиторий).
- Содержит шаблоны для исключения временных файлов IDE (IntelliJ IDEA, Eclipse, VS Code), сборочных артефактов (out/, bin/), служебных файлов ОС (.DS_Store на Mac) и других.

2. KotlinProgram.iml

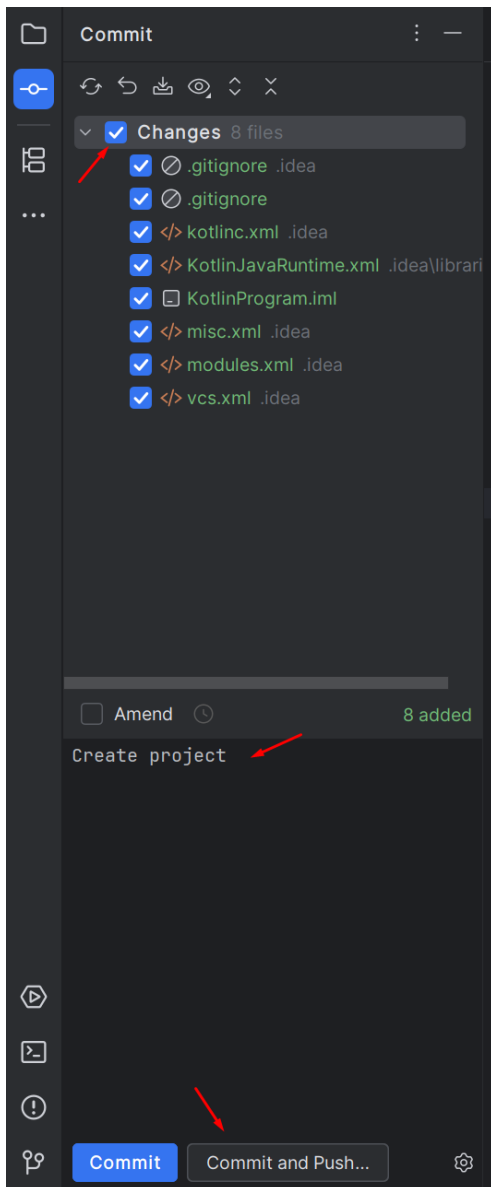
- Файл модуля IntelliJ IDEA (IML = IntelliJ Module).
- Хранит настройки проекта:
 - Структуру папок (исходный код src/, тесты test/, ресурсы).
 - Зависимости (например, KotlinJavaRuntime — библиотека Kotlin для JVM).
 - Настройки SDK (JDK).
- Автоматически генерируется IDE, обычно не редактируется вручную.

Шаг 2. Создаём коммит

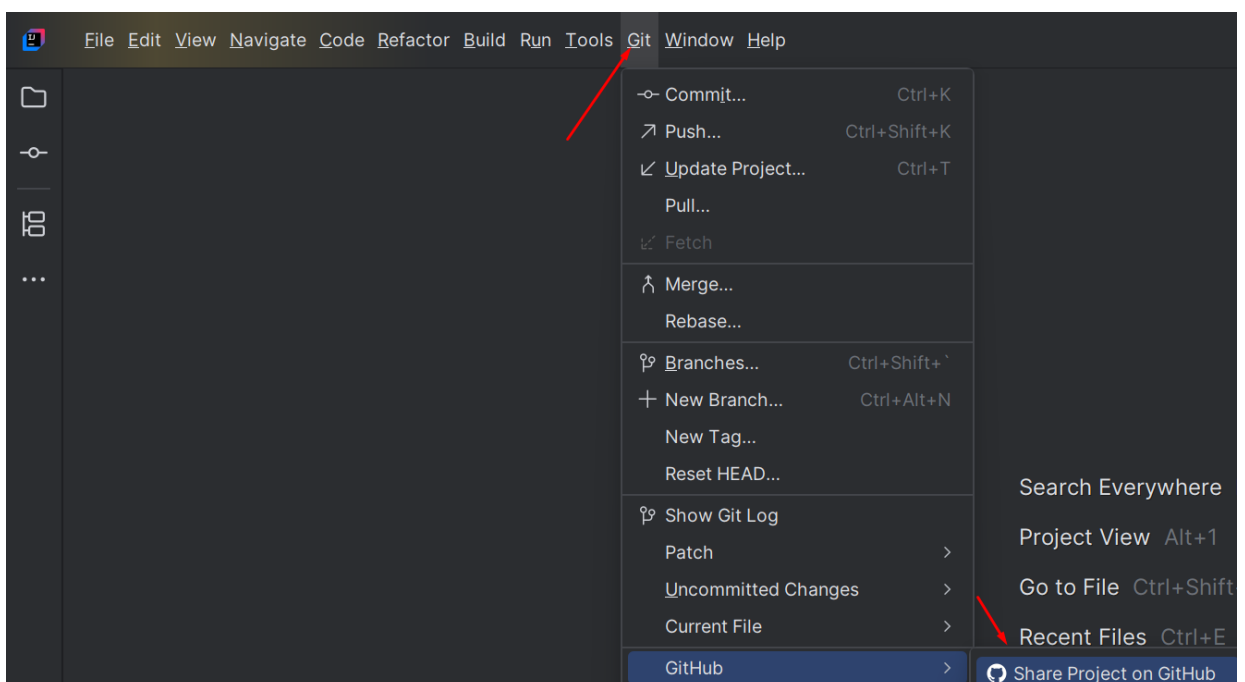
Включите отображения коммитов, перейдя в **View – Tool Windows – Commit**:



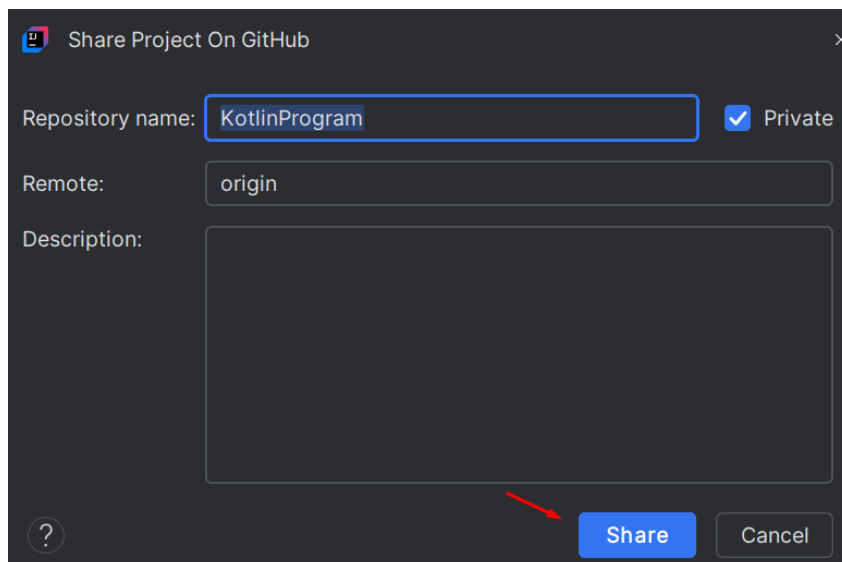
Поставьте везде галочки, введите в текст **Create project** выберите пункт **Commit and Push**:



Дальше нажмите **Git – GitHub – Share Project on GitHub:**

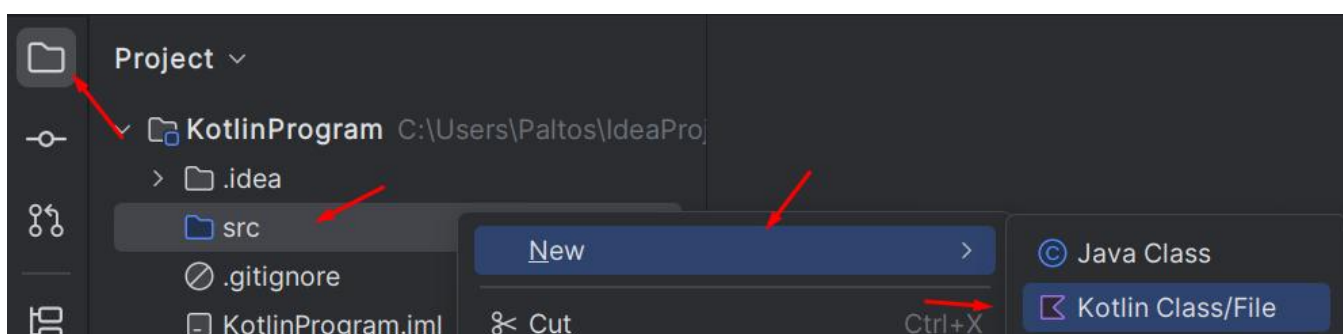


Нажмите **Share:**

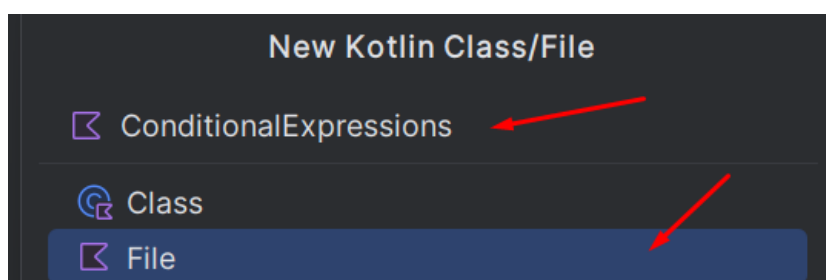


Шаг 3. Создаём точку входа и коммитим

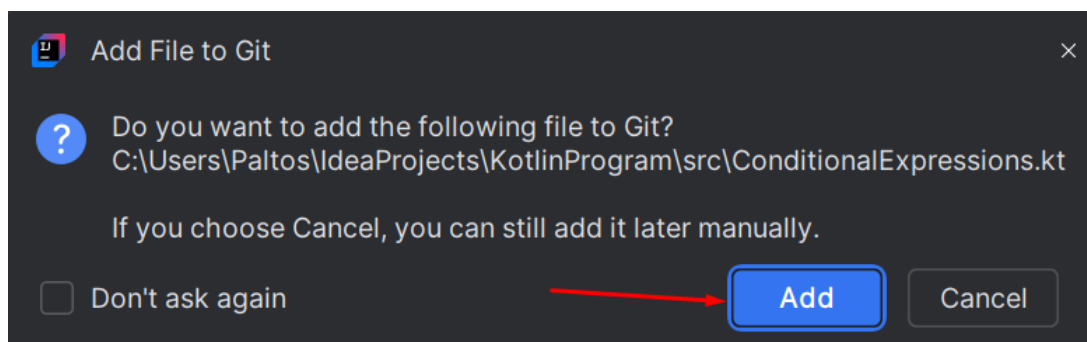
Щёлкните ПКМ по папке **src** и выберите **New – Kotlin Class/File**:



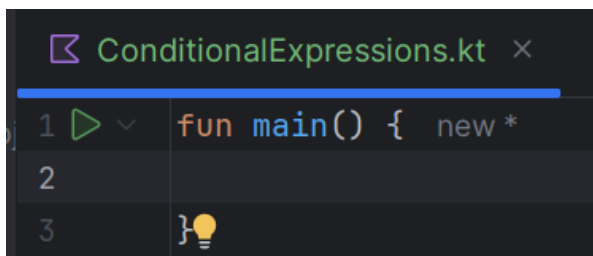
Так как мы начнём изучать условные выражения, то введите имя **ConditionalExpressions** и выберите **File**:



Нажмите добавить в **Git**:



Создадим точку входа в нашу программу (простой способ начать вводить **main** и нажать клавишу **TAB**):



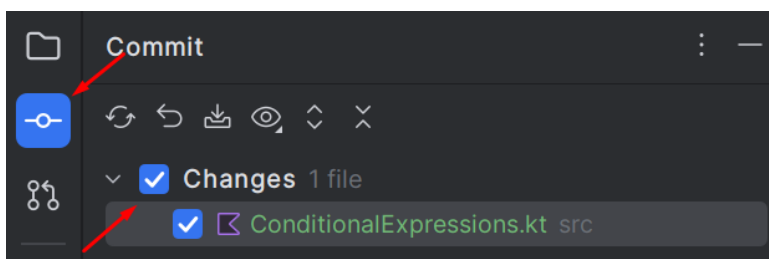
```
ConditionalExpressions.kt x
1 fun main() { new *
2
3 }
```

И выведите в строку вывода **Hello world** и запустим нашу программу:

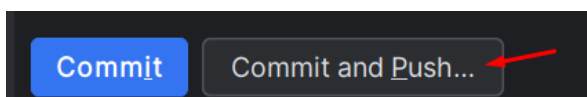


```
ConditionalExpressions.kt x
1 fun main() { new *
2     println("Hello world!")
3 }
```

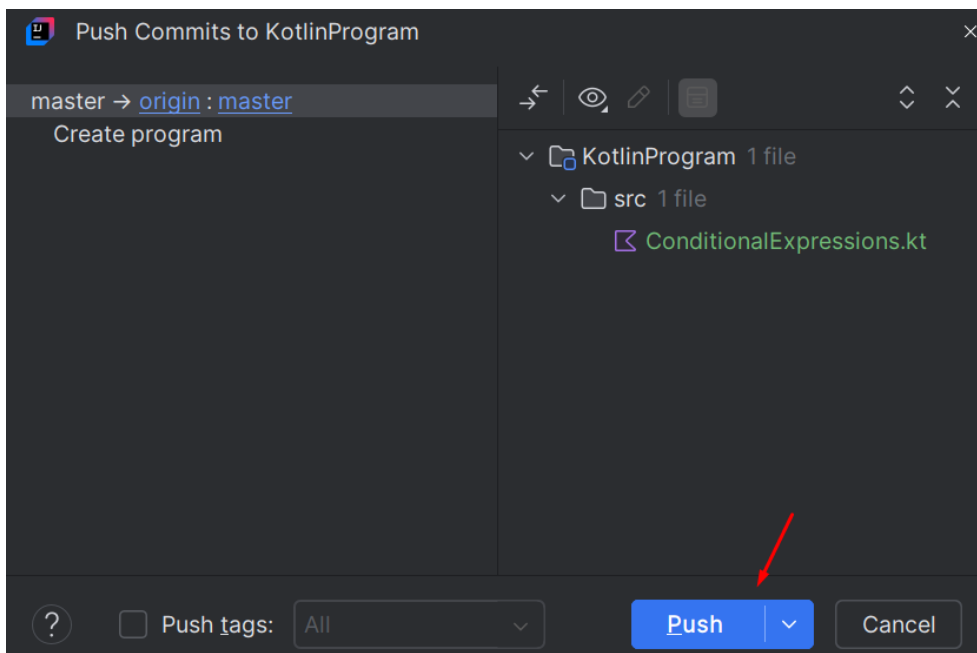
Теперь давайте создадим **коммит** (походу лабораторной работы старайтесь создавать после решения каждого задания). Нажмите на кнопку (или сочетание клавиш **Alt+0**), поставьте галочку на изменениях, введите **название коммита** (например **Create program**), и нажмите **Commit and Push**:



Create program



Ещё раз **Push**:



Шаг 4. Условные выражения

В Kotlin `if/else` — это **выражение** (expression), которое возвращает значение, в отличие от Python, где это **инструкция** (statement). Это ключевое отличие позволяет писать более лаконичный код.

Давайте напишем простой пример, который сравнивает два числа (не забудьте сделать коммит программы):

```
fun main() { Paltosik92 *
    print("Введите первое число: ")
    val number1 = readln().toInt()
    print("Введите второе число: ")
    val number2 = readln().toInt()

    if (number1 > number2) {
        println("Число $number1 больше чем число $number2")
    } else {
        println("Число $number2 больше чем число $number1")
    }
}
```

Для удобного форматирования кода нажимайте сочетание клавиш **Ctrl+Alt+L**

Итак, первое что мы можем с вами сделать это вернуть значение `if` (сохранить в переменную):

```
7      val max = if (number1 > number2) number1 else number2
8      println("Максимальное число - $max")
9  }
```

В Kotlin нет тернарного оператора `a ? b : c`, потому что **if/else** его заменяет (но он есть, например в C#). Запишем второй пример:

```
fun main() { Paltosik92 *
    val age = readln().toInt()
    val status = if (age >= 18) "Взрослый" else "Ребёнок"
    println(status)
}
```

Мини-задание. Магический щит (защита от урона)

Задача:

Игрок имеет щит (**shieldActive**). Если щит активен, урон уменьшается вдвое.

Пример реализации задачи:

```
Введите урон: 67
Щит активен? (введите да или нет): да
Полученный урон: 33
```

Теперь давайте рассмотрим пример с логическими операторами **&&** (И) и **||** (ИЛИ).

Задача: "Доступ в закрытую зону игры"

Условие: Игрок хочет попасть в закрытую зону. Для этого нужно:

- **Уровень игрока ≥ 30** (Int ввод), **И**
- **Есть VIP-статус** (Boolean ввод: true/false).

Но! Если у игрока **уровень ≥ 50** , VIP-статус не требуется (доступ открыт просто по уровню).

Требуется написать программу, которая:

1. Запрашивает уровень игрока.
2. Спрашивает, есть ли у него VIP-статус (true/false).
3. Проверяет условия и выводит:
 - "Доступ разрешен!" если условия выполнены.
 - "Доступ запрещен!" если нет.

Давайте пошагово решим её. Вначале мы должны запросить уровень игрока и сохранить в переменную:

```
print("Введите ваш уровень: ")
val level = readln().toInt()
```

Затем нам нужно узнать есть ли у него доступ к VIP, при этом сохранить ответ в виде булевой переменной:

```
print("У вас есть VIP-статус? (true/false): ")
val hasVIP = readln().toBoolean()
```


1. **Условие 1:** (level >= 30 && hasVIP)

- && (**И**) — доступ есть, только если **оба** условия верны:
 - Уровень ≥ 30 **И** VIP-статус есть.

2. **Условие 2:** level >= 50

- Если уровень ≥ 50 , доступ открыт **без VIP** (благодаря ||).

3. || (**ИЛИ**) — общее условие срабатывает, если:

- Выполнено **Условие 1 ИЛИ Условие 2**.

```
if ((level >= 30 && hasVIP) || level >= 50) {  
    println("Доступ разрешен!")  
}
```

В ином случае запрещаем вход:

```
} else {  
    println("Доступ запрещен!")  
}
```

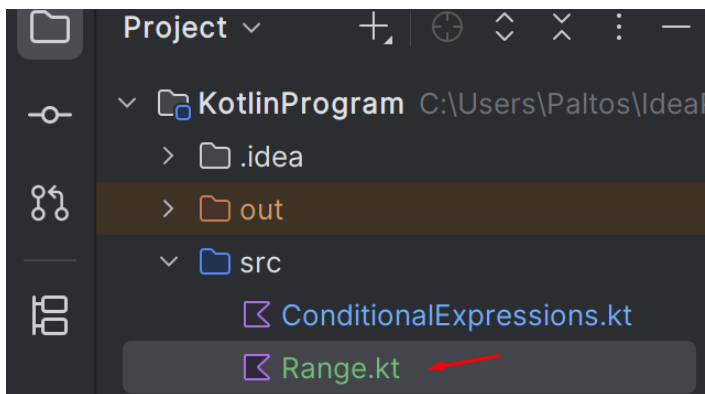
Итоговый код программы выглядит следующим образом:

```
print("Введите ваш уровень: ")  
val level = readln().toInt()  
print("У вас есть VIP-статус? (true/false): ")  
val hasVIP = readln().toBoolean()  
  
if ((level >= 30 && hasVIP) || level >= 50) {  
    println("Доступ разрешен!")  
} else {  
    println("Доступ запрещен!")  
}
```

Шаг 5. Диапазоны

Диапазоны — это интервалы значений (чисел, символов, дат), которые часто используются в циклах, проверках условий и генерации последовательностей.

Создайте новый файл для работы и назовите его **Range**:



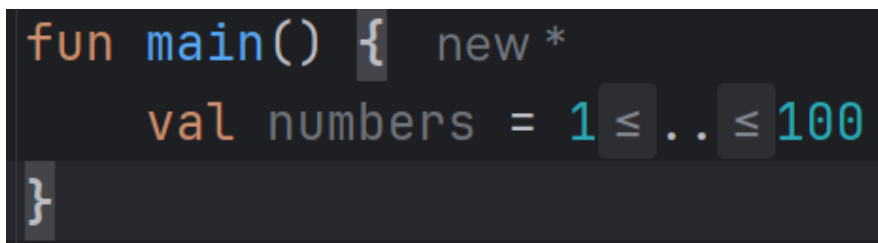
Объявите точку входа **main**:



Напишите диапазон от 1 до 100 (знаки ставить не нужно, это IDEA показывает нам):

val numbers = 1..100

Итог:

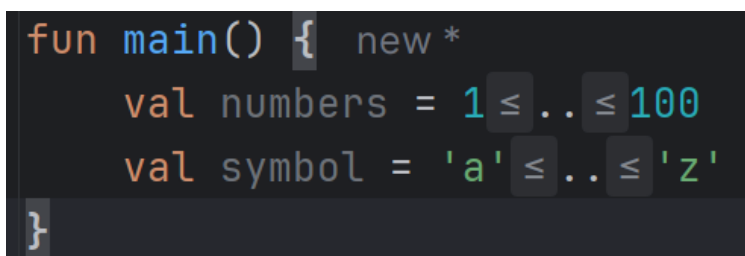


Обратите внимание, что мы можем указывать не только числовые диапазоны, они бывают следующих видов:

1. **Числовые** (Int, Long, Double)
2. **Символьные** (Char)
3. **Пользовательские** (для своих классов через оператор ..).

В качестве примера установите диапазон от a до z:

val symbol = 'a'..'z'



С помощью специальных операторов можно проверить наличие или отсутствие элементов в диапазоне:

- **in**: возвращает true, если объект имеется в диапазоне
- **!in**: возвращает true, если объект отсутствует в диапазоне

В качестве примера давайте решим задачу, где пользователь вводит уровень персонажа, и если он входит в диапазон от 1 до 50, то мы можем войти в данж.

Попробуйте решить задачу самостоятельно.

В начале мы спрашиваем у пользователя его уровень:

```
print("Введите ваш уровень: ")
```

После сохраняем его в переменную:

```
val level = readln().toInt()
```

Дальше, используя условие if, мы проверяем входит ли наш уровень (переменная) в диапазон от 1 до 50, и выводим на консоль что доступ пользователю разрешён:

```
if (level in 1..50) {  
    println("Уровень $level соответствует требованиям! Вы можете зайти в данж!")  
}
```

В ином случае запрещаем вход:

```
} else {  
    println("Высокий уровень! Вход запрещён!")  
}
```

В диапазонах можно использовать операторы:

Оператор	Пример	Описание
..	1..10	Включительно ($1 \leq x \leq 10$)
until	1 until 10	Полуоткрытый ($1 \leq x < 10$)
downTo	10 downTo 1	Убывающий (10, 9, ..., 1)
step	1..10 step 2	Шаг (1, 3, 5, ..., 9)

В качестве примера создайте диапазоны символов в порядке от **z** до **a**:

```
z y x w v u t s r q p o n m l k j i h g f e d c b a
```

Диапазон от 5 до 11 (не включая верхний диапазон 11):

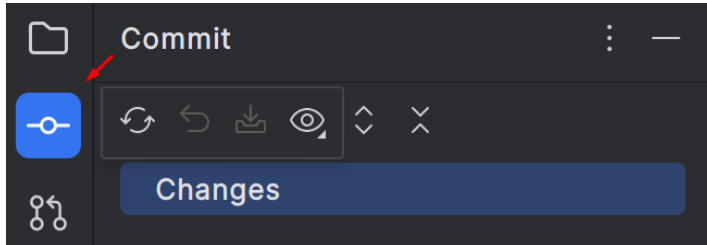
```
5 6 7 8 9 10
```

Диапазон от 10 до 50 с шагом 3:

10 13 16 19 22 25 28 31 34 37 40 43 46 49

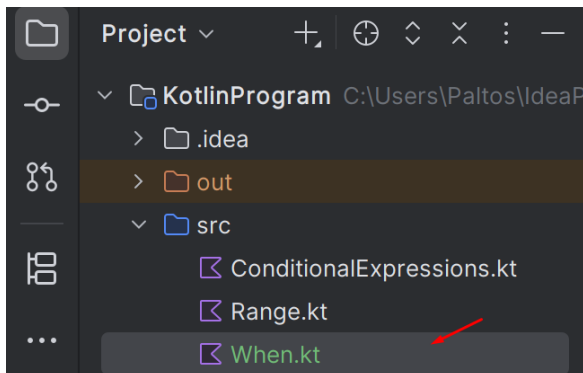
Лучше всего диапазоны используются с циклами, и мы рассмотрим их в будущих примерах.

Создайте **КОММИТ** вашего проекта:

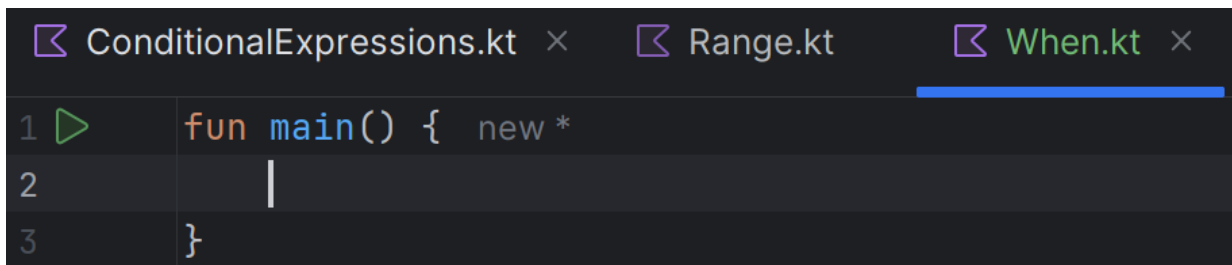


Шаг 6. Конструкция **when**

Создайте новый файл для работы и назовите его **When**:



Объявите точку входа **main**:



Конструкция **when** проверяет значение некоторого объекта и в зависимости от его значения выполняет тот или иной код. Конструкция **when** аналогична конструкции **switch** (например, в C#) в других языках.

Для разбора решите следующую задачу используя конструкцию **if/else**:

Программа спрашивает у пользователя порядковый номер месяца, программа выводит «Вы ввели порядковый номер X, ему соответствует месяц N.»:

```
Введите порядковый номер месяца (1-12): 2
Вы ввели порядковый номер 2, ему соответствует месяц Февраль.
```

Попробуйте решить задачу самостоятельно.

Ввод данных

- Программа запрашивает у пользователя номер месяца (от 1 до 12):

```
print("Введите порядковый номер месяца (1-12): ")
```

- Значение сохраняется в переменную **index** (тип Int).

```
val index = readln().toInt()
```

- Создаём пустую переменную **month**, чтобы затем сохранить в ней месяц:

```
var month = ""
```

Определение месяца

- С помощью цепочки if-else проверяется значение index.
- В зависимости от числа, переменной month присваивается соответствующее название месяца.

```
if (index == 1) {  
    month = "Январь"  
} else if (index == 2) {  
    month = "Февраль"  
} else if (index == 3) {  
    month = "Март"  
} else if (index == 4) {  
    month = "Апрель"  
} else if (index == 5) {  
    month = "Май"  
}
```

```
else if (index == 6) {  
    month = "Июнь"  
} else if (index == 7) {  
    month = "Июль"  
} else if (index == 8) {  
    month = "Август"  
} else if (index == 9) {  
    month = "Сентябрь"  
} else if (index == 10) {  
    month = "Октябрь"  
} else if (index == 11) {  
    month = "Ноябрь"  
} else if (index == 12) {  
    month = "Декабрь"  
}
```

Вывод результата

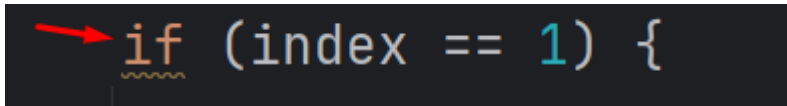
- Если номер в диапазоне 1..12, выводится сообщение с номером и названием месяца.

```
if (index in 1..12) {  
    println("Вы ввели порядковый номер $index, ему  
    соответствует месяц $month.")  
}
```

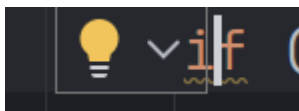
Мы видим, что у текущей реализации есть проблемы:

1. **Дублирование кода:** Много однотипных условий if-else.
2. **Нет обработки неверного ввода:** Если ввести число вне 1..12, программа молча завершится (переменная month останется пустой).
3. **Неочевидность:** Цепочка if-else из 12 условий выглядит громоздко.

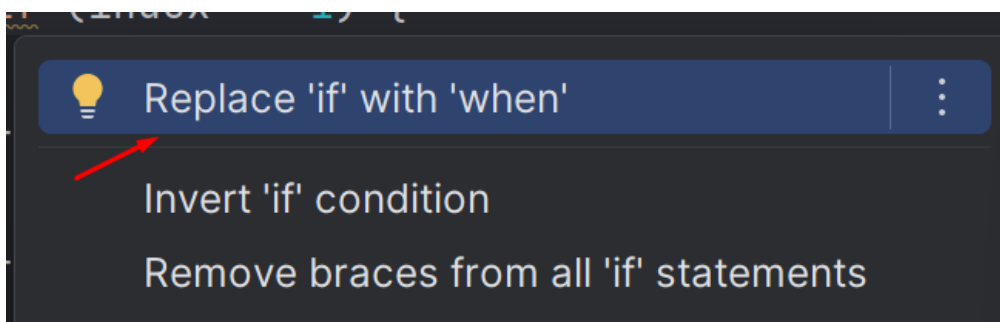
Давайте с вами улучшим программу. Если обратите внимание оператор **if** подчёркивается линией в IDE:



Нажмите на **if**, и используйте сочетание клавиш **Alt+Enter** (другой вариант навести на лампочку):

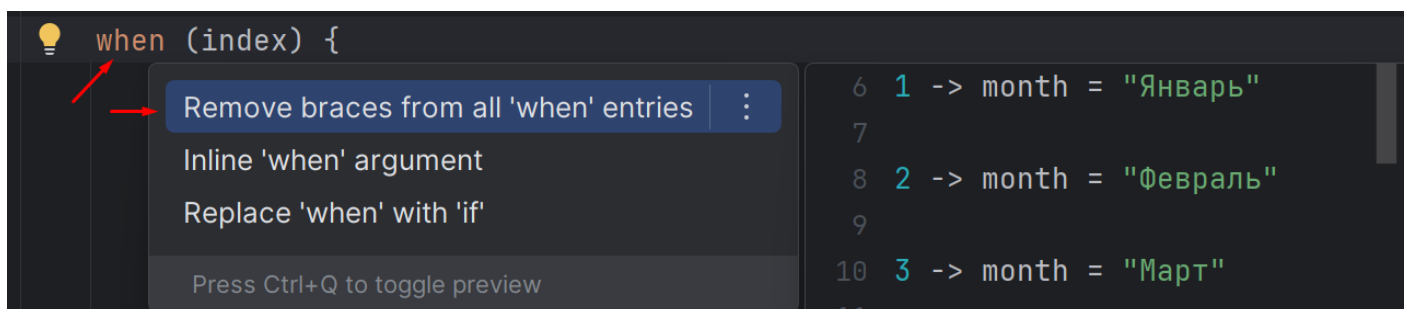


Выберем вариант отформатировать в конструкцию **when**:



Затем нажмите **Ctrl+Alt+L** чтобы сделать форматирование кода. Вы увидите, что у вас получилась очень большая конструкция, которую можно было бы вручную сократить, удалив ненужные пробелы и разрывы строк. Но давайте также используем возможности **IntelliJ IDEA**.

Нажмём на оператор **when**, и выберем пункт **Remove braces from all 'when' entries** **entries**:



Удаляем ненужные пробелы и видим более лаконичную конструкцию:

```

when (index) {
  1 -> month = "Январь"
  2 -> month = "Февраль"
  3 -> month = "Март"
  4 -> month = "Апрель"
  5 -> month = "Май"
  6 -> month = "Июнь"
  7 -> month = "Июль"
  8 -> month = "Август"
  9 -> month = "Сентябрь"
  10 -> month = "Октябрь"
  11 -> month = "Ноябрь"
  12 -> month = "Декабрь"
}

```

Нам нужно добавить в эту конструкцию обработку неверного ввода через **else**:

```

12 -> month = "Декабрь"
else -> "некорректный номер"
}

```

Также если вы помните, то условия мы можем сохранять в переменную, поэтому можем заменить конструкцию (для более быстрого выделения однотипных слов, выберите нужное вам слово и через **Alt+J** выделите остальные слова, далее через **Delete** удалите их):

```

var month = ""
when (index) {
  1 -> month = "Январь"
  2 -> month = "Февраль"
  3 -> month = "Март"
  4 -> month = "Апрель"
  5 -> month = "Май"
  6 -> month = "Июнь"
  7 -> month = "Июль"
  8 -> month = "Август"
  9 -> month = "Сентябрь"
  10 -> month = "Октябрь"
  11 -> month = "Ноябрь"
  12 -> month = "Декабрь"
  else -> "некорректный номер"
}

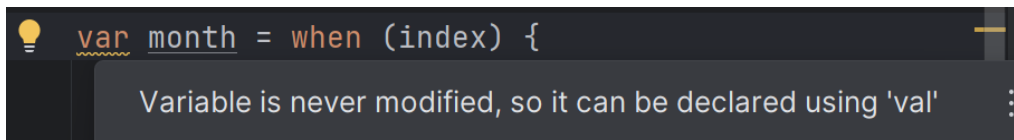
```

```

var month = when (index) {
  1 -> "Январь"
  2 -> "Февраль"
  3 -> "Март"
  4 -> "Апрель"
  5 -> "Май"
  6 -> "Июнь"
  7 -> "Июль"
  8 -> "Август"
  9 -> "Сентябрь"
  10 -> "Октябрь"
  11 -> "Ноябрь"
  12 -> "Декабрь"
  else -> "некорректный номер"
}

```

Обратите внимание, что у нас подчёркивается значение переменной `var`, наведите и прочитайте:



Мы сейчас один раз присваиваем значение результата выражения, поэтому лучше использовать **val** (и везде, где это возможно в ваших программах используйте `val`). Поэтому замените в ваше коде:

```
val month = when (index) {
```

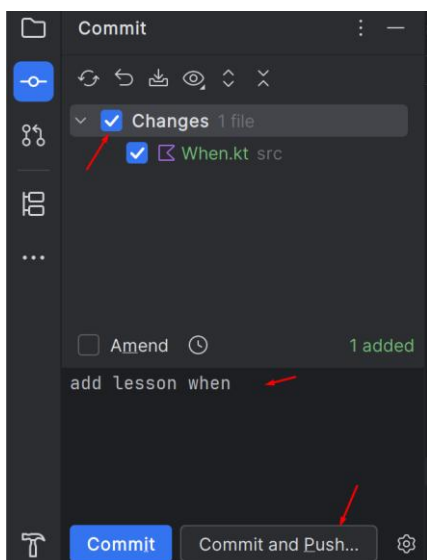
? Почему `val` лучше `var` в этом случае?

1. **Избегание мутаций.** Если переменная не должна меняться после создания, `val` предотвращает случайные изменения.
2. **Читаемость.** Понятно, что `month` вычисляется **однократно** на основе `index`.
3. **Идиоматичный Kotlin.** В Kotlin предпочитают `val` везде, где возможно. Это рекомендация официального стиля.
4. **Потокобезопасность.** Неизменяемые данные (`val`) безопасны при работе с многопоточностью.

И последняя проблема, которую мы должны решить в нашей программе, добавим вывод неверного ввода:

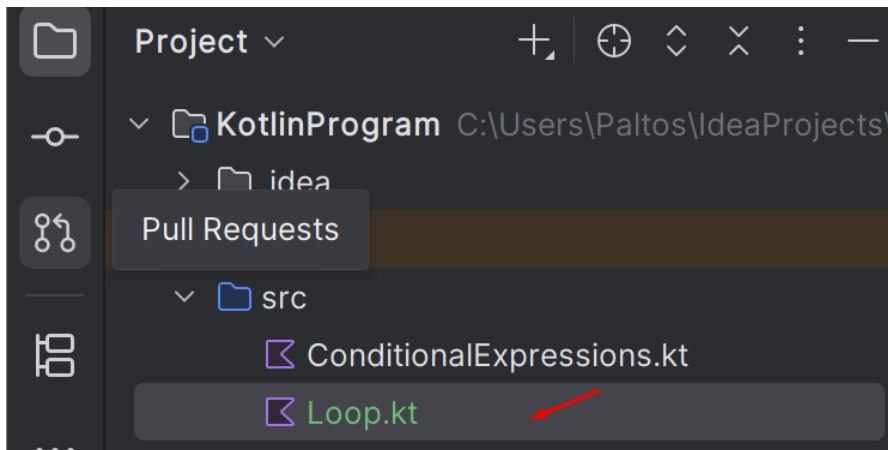
```
if (index in 1..12) {  
    println("Вы ввели порядковый номер $index,  
            ему соответствует месяц $month.")  
} else {  
    println("Ошибка: месяц с номером $index не  
            существует!")  
}
```

Создайте **коммит** вашего проекта:

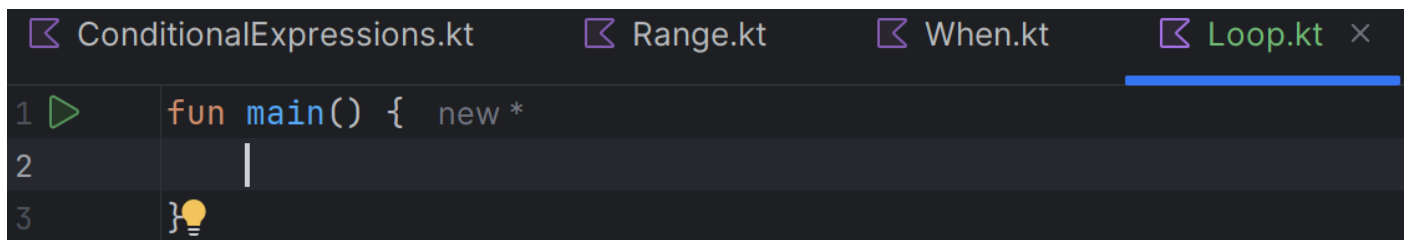


Шаг 7. Цикл repeat

Создайте новый файл для работы и назовите его **Repeat**:



Объявите точку входа **main**:



Цикл **repeat** – это цикл, который повторяет блок кода заданное количество раз.

Аналог `for _ in range(N)` в Python, но проще и более читаемый.

Его синтаксис:

```
repeat( times = количество_повторов ) {  
    // Тело цикла (повторяемый код)  
}
```

Давайте рассмотрим на простом примере. У нас есть задача:

Маг начинает готовить ритуал защиты замка, затем произносит заклинание защиты 5 раз и после мы устанавливаем защиту.

Вставьте следующий код в `main`:

```
println("Начинаем ритуал защиты!")  
println("Заклинание защиты активировано!")  
println("Заклинание защиты активировано!")  
println("Заклинание защиты активировано!")  
println("Заклинание защиты активировано!")  
println("Заклинание защиты активировано!")  
println("Замок защищен!")
```

```
fun main() { new *
    println("Начинаем ритуал защиты!")
    println("Заклинание защиты активировано!")
    println("Заклинание защиты активировано!")
    println("Заклинание защиты активировано!")
    println("Заклинание защиты активировано!")
    println("Заклинание защиты активировано!")
    println("Замок защищен!")
}
```

Согласитесь, что это писать довольно долго, даже если мы можем копировать однотипные строчки кода. Поэтому мы можем с вами использовать цикл для повтора наших действий 5 раз. Перепишите код:

```
fun main() { new *
    println("Начинаем ритуал защиты!")
    repeat(times = 5) {
        println("Заклинание защиты активировано!")
    }
    println("Замок защищен!")
}
```

Теперь давайте улучшим код, добавив отображение текущего шага!

Создаём переменную счётчик **index**, используя **var** (потому что наша переменная будет изменяться):

```
println("Начинаем ритуал защиты!")
var index = 0
```

Меняем конструкцию в цикле:

```
repeat(times = 5) {
    println("Заклинание защиты активировано! (Шаг ${index + 1})")
    index++ // Увеличиваем счётчик на 1 после каждого шага
}
```

В цикле в строке **\${index + 1}** мы добавляем +1, чтобы шаги считались с 1, а не с 0.

`index++` — это сокращённая запись для `index = index + 1`.

Мы также можем ещё улучшить нашу конструкцию, для этого в Kotlin можно обойтись без ручного счётчика, используя встроенный параметр `it`:

```
println("Начинаем ритуал защиты!")

repeat(times = 5) { index ->
    println("Заклинание защиты активировано! (Шаг ${index + 1})")
}

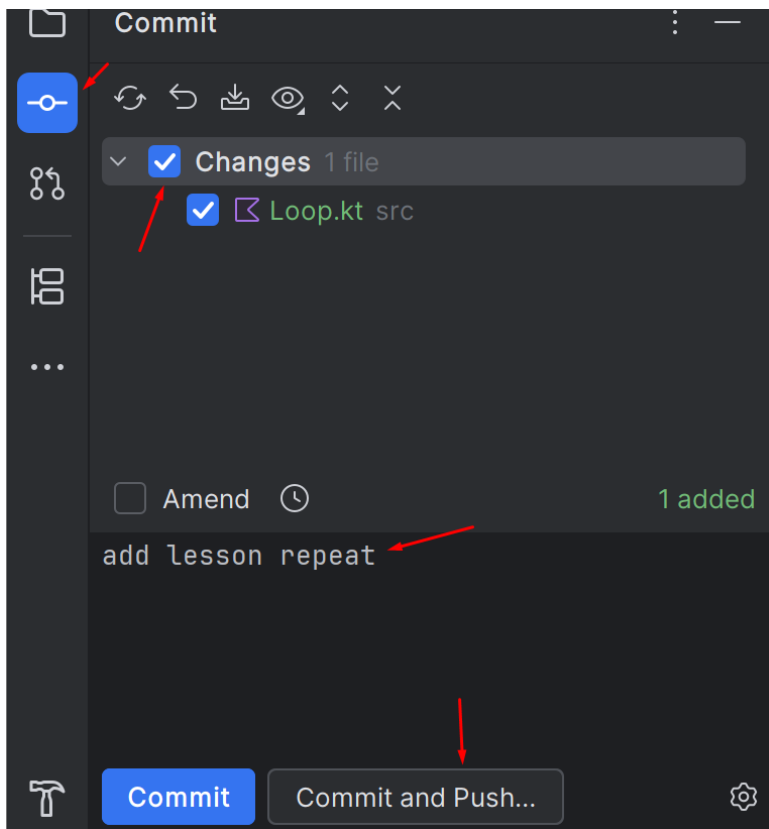
println("Замок защищен!")
```

Данный цикл является самым простым в Kotlin, но у него есть и ограничения:

- Не поддерживает break и continue (в отличие от for/while).
- Если нужно прервать — используйте return@repeat (но это неочевидно).

```
repeat(times = 5) { index ->
    if (index == 2) {
        println("Пропускаем шаг ${index + 1} из-за магического вмешательства!")
        return@repeat // Пропускаем эту итерацию
    }
    println("Заклинание защиты активировано! (Шаг ${index + 1})")
}
```

Создайте **КОММИТ** вашего проекта:



Шаг 8. Введение в коллекции

listOf — это **неизменяемый список** (immutable list), то есть мы **не можем** добавлять или удалять элементы после создания, но можем **читать** и **получать доступ по индексу**.

Создайте простую коллекцию:

```
val months = listOf("Python", "Kotlin", "C#", "C++", "Java")
```

Мы можем получить элемент по индексу (Индексация начинается с 0):

```
println(months[0])
```

Узнать размер списка:

```
println(months.size)
```

Проверить наличие элемента

```
println("Kotlin" in months)
```

На основе изученных тем решим следующую задачу:

Пользователь вводит номер месяца (от 1 до 12), а программа выводит количество дней в этом месяце. **Без високосного года.**

```
Введите номер месяца (1-12): 12
В этом месяце 31 дней.
```

Итак, вначале мы должны завести список, который содержит количество дней по месяцам:

```
val daysInMonth = listOf(31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)
```

Затем мы спрашиваем у пользователя номер месяца:

```
print("Введите номер месяца (1-12): ")
```

Сохраняем ввод в отдельную переменную:

```
val input = readln().toInt()
```

И выполняем проверку с выводом:

```
if (input in 1 ≤ .. ≤ 12) {
    val days = daysInMonth[input - 1]
    println("В этом месяце $days дней.")
} else {
    println("Некорректный номер месяца.")
}
```

Мы обращаемся к элементу списка через `input - 1`, потому что индексация начинается с 0.

Шаг 9. Символьный тип данных

Каждый символ — это просто символ, заключенный в одинарные кавычки. **Char** тип представляет собой буквы (как заглавные, так и строчные), цифры и другие символы:

```
fun main() {
    val lowerCaseLetter: Char = 'a'
    val upperCaseLetter: Char = 'Q'
    val number: Char = '1'
    val space: Char = ' '
    val dollar: Char = '$'
}
```

Этот тип может представлять любой символ, включая иероглифы, а также некоторые специальные символы.

Символ также может быть представлен его шестнадцатеричным кодом в таблице Unicode. Код начинается с `\u`:

```
val ch = '\u0040' // символ '@'
println(ch) // @
```

Хотя для представления такого кода мы используем последовательность символов, сам код представляет собой ровно один символ.

Например, заглавные латинские буквы имеют шестнадцатеричные коды от '\u0041' до '\u005A', а строчные латинские буквы имеют коды в диапазоне от '\u0061' до '\u007A'.

Вы также можете преобразовывать числа в символы и наоборот. Давайте посмотрим, как это работает:

```
val ch = 'a'
println(ch.code)    // 97
val num = 97
println(num.toChar()) // a
```

Kotlin не предоставляет удобной функции для чтения Char. Однако вы можете сделать это по-другому: если вам нужно прочесть один Char в целой строке, используйте эту конструкцию:

```
val ch: Char = readln().first()
```

Вы просто считываете String и получаете его первый элемент, который будет Char (строки состоят из символов).

Существует два оператора для сложения (+) и вычитания (-) целых чисел с целью получения следующего и предыдущего символов в соответствии с порядком Unicode:

```
val ch1 = 'b'
val ch2 = ch1 + 1 // 'c'
val ch3 = ch2 - 2 // 'a'
```

Помните, что добавлять символ к числу нельзя, так как это приведет к ошибке.

```
val ch = 'a' 1 Usage
val ch1 = 1 + ch // Error
```

Также нельзя суммировать два символа:

```
val charsSum = 'a' + 'b' // Error
```

Можно использовать операторы инкремента (++) и декремента (--) в их префиксной и постфиксной формах. Оператор присваивания в сочетании с + или - также работает для символов, а также += и -=.

```
var ch = 'A'
ch += 10
println(ch)    // 'K'
println(++ch)  // 'L'
println(++ch)  // 'M'
println(--ch)  // 'L'
```

Существуют некоторые специальные символы, начинающиеся с обратной косой черты \, которые известны как **escape-последовательности** или **управляющие последовательности**. Большинство из них не имеют соответствующих символов, и вы не можете найти их на своей клавиатуре. Для представления таких символов мы используем пару обычных символов. В программе эта пара рассматривается как ровно один символ с соответствующим кодом.

- '\n' символ новой строки;
- '\t' символ табуляции;
- '\r' символ возврата каретки;
- '\\' это сам символ обратной косой черты;
- '\"' одинарная кавычка;
- '\"' это двойные кавычки.

Вы можете сравнивать символы, используя реляционные операции (==, <, >, <=, >= и !=) в соответствии с их положением в таблице Unicode.

```
println('a' < 'c') // true
println('x' >= 'z') // false
println('D' == 'D') // true
println('Q' != 'q') // true
println('A' < 'a') // true
```

Существует множество полезных функций для работы с символами. Вы можете использовать эти функции вместо того, чтобы иметь дело с кодами символов.

- **isDigit()** возвращает, true если заданный символ представляет цифру ('1', '2', и т.д.); в противном случае, false;
- **isLetter()** возвращает, true если заданный символ представляет букву ('a', 'B', 'm', и т.д.); в противном случае, false;

- **isLetterOrDigit()** возвращает, true если заданный символ представляет собой букву или цифру; в противном случае, false;
- **isUpperCase()** возвращает, true если заданный символ является заглавным; в противном случае, false;
- **isLowerCase()** возвращает, true если заданный символ является строчным; в противном случае, false;
- **uppercaseChar()** возвращает заглавную форму указанного символа (начиная с Kotlin 1.5);
- **uppercase()** возвращает a Stringc заглавной формой указанного символа (начиная с Kotlin 1.5);

Давайте рассмотрим некоторые примеры перечисленных выше функций:

```
val one = '1'
val isDigit = one.isDigit()    // true
val isLetter = one.isLetter()  // false
val capital = 'A'
val small = 'e'
val isLetterOrDigit = capital.isLetterOrDigit() // true
val isUpperCase = capital.isUpperCase() // true
val isLowerCase = capital.isLowerCase() // false
val capitalEString = small.uppercase() // "E"
val capitalE = small.uppercaseChar()    // 'E'
```

Самостоятельные задания

Задание 1. Проверка уровня доступа игрока

Задача: Игрок вводит свой уровень (level).

- Если уровень < 10, он "Новичок" и получает бонус +10 НР.
- Если $10 \leq \text{level} < 50$, он "Опытный" и получает +30 НР.
- Если ≥ 50 , он "Легенда" и получает +100 НР.

Примера вывода:

```
Введите ваш уровень: 51
Легенда! Вы получаете +100 НР.
```


Введите ваш уровень: 8
Новичок! Вы получаете +10 НР.

Задание 2. Проверка возраста

Запросите возраст пользователя с клавиатуры. Выведите:

- "Вы ребёнок", если возраст меньше 12;
- "Вы подросток", если от 12 до 17 включительно;
- "Вы взрослый", если больше 17.

Используйте конструкцию if-else.

Задание 3. Проверка диапазона

Попросите пользователя ввести число от 1 до 100. Выведите, входит ли оно в диапазон от 10 до 50 включительно. Используйте оператор in.

Задание 4. Распознавание символа

Пусть пользователь введёт один символ. Определите, является ли он:

- Цифрой
- Буквой латинского алфавита
- Пробелом
- Спецсимволом

Используйте конструкцию when и диапазоны: 'a'..'z', '0'..'9'.

Задание 5. Таблица умножения

Выведите таблицу умножения для числа 5, используя цикл repeat.
Формат вывода: 5 x 1 = 5, 5 x 2 = 10, ... до 5 x 10.

Задание 6. Коллекция любимых фильмов

- Создайте неизменяемую коллекцию из 5 названий фильмов.
- Выведите все названия на экран по одному.
- Выведите длину названия каждого фильма.

Используйте listOf() и цикл for.

Задание 7. Создайте программу-миниопрос

1. Спросите имя, возраст, любимый цвет.
2. Постройте короткий ответ типа:

Привет, Иван! Тебе 21 год, и ты любишь синий.

Это задание помогает закрепить шаблоны строк.